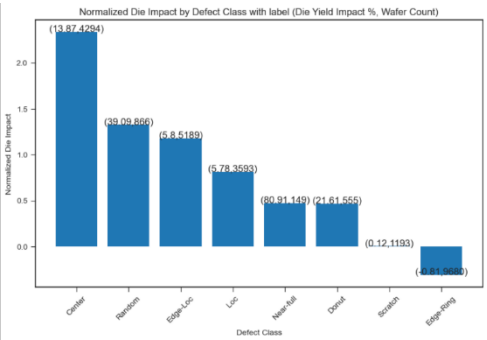
An Analysis of Wafer Defect Patterns to Assess Yield Impact, Prevalence, and Automate Wafer Defect Classification

Kaustubh Indane

Executive Summary

Defect classification in semiconductor manufacturing is a costly labor-intensive process that requires engineers to manually inspect and categorize wafer defects. Classification is key to address systematic issues promptly, increase yield on product, and avoid expensive delays to design process improvements. To address this challenge, we have developed an automatic defect classification model to categorize known defect patterns based on manually classified data. Industry benchmarks suggest that automating process can reduce defect review efforts by 60-80%, translating to estimated annual savings between $500K and $2M (Averroes). Through EDA, we will also characterize the impact/prevalence of defect patterns to determine priority for resolutions, quantify impact on yield, and assist in root cause determination to tools/process changes.

After accommodating for edge partial dies with zero yields, EDA revealed a baseline yield loss of 8.17%, illustrating process margin for improvement outside defect patterns. Center defects had the highest prevalence and penalized wafer yield by 13.9% (Figure 1). Random had second highest importance per normalized metrics with per wafer yield loss of ~39%. More work needs to be done to manually classify defects in this bucket. Scratch, Edge-Ring, Loc, and Donut defects have lowest impact on yield and should be deprioritized for investigations into root cause. Additionally, wafers positioned in extreme/marginal positions in processing lots exhibited increased defect rates, indicating the need to 'buffer' lots with blank wafers on the ends. We also identified processing window between lot 26400 – 32400 where Edge-Ring defects were most prevalent to assist in root cause investigation.



Recommendation is to utilize the Custom CNN model for automatic defect classification implementation which has a validated accuracy of 94%. Even with this model, we will misclassify ~25% of Loc, Scratch, and Edge-Loc defects as 'none.' Voting emsemble methods with models outside this study can improve classification of these labels.

Ultimately, we want to use RGB data to uniquely identify morphologies of defects and improve classification on an individual defect level instead of just defect patterns. Further studies into MobileNetv2 implementation should be studied as it will provide outstanding performance in that case.

Problem Statement/Research Objectives

Semiconductor manufacturing relies on defect and variation reduction to improve performance, quality, and yield of products. However, defect identification and classification are traditionally labor-intensive processes that require significant engineering time for manual review and analysis. This results in increased costs, slower decision-making, and expensive delays in yield improvement. This study aims to develop a wafer defect classification model with machine learning that identifies and categorizes defect patterns from wafer map defectivity data. By performing EDA, we will characterize the impact and prevalence of defect patterns to determine priority for resolving defects, quantify impact on yield/profits, and assist in root cause determination to tools/process changes. These insights allow engineers to take proactive measures, optimizing manufacturing processes and improving yield. The dataset is from Multimedia Information Retrieval Lab at National Taiwan University who focus on audio/visual recognition, retrieval, and synthesis. 'MIR-WM811K' dataset is a collection of real-world wafer defect maps from an unspecified manufacturing fabrication in Taiwan.

We aim to answer the following initial descriptive statistics questions through EDA:

1. What types of defects are most prevalent?
2. What defects have the highest impact on die yield?
3. What is the baseline level of defects/yield of every wafer?
4. Is there a Slot dependency to any defects or patterns? (25 wafers are processed in a lot batch)
5. Is there any time period significant to a defect or pattern? Do any defect patterns have a "turn on" date? (This will assist in root cause determination for process engineers)

We aim to build an automatic defect classification model by:

1. Extracting density, geometric, radial features from defectivity data to uniquely classify defect patterns (**Defect_Class = Scratch, Donut, Center, Edge-Ring, Edge-Loc, Loc, Random, Near-full, None**)
2. Create k-NN, Random Forest, custom CNN, and MobileNet-v2 models for classification with high accuracy and low computation time
3. Consider an ensemble approach for decision-making
4. Explore performance of model results with validity, generalizability, and metrics, including confusion matrices, accuracy, precision, recall, F1, AUC
5. Define business improvement from automation of defect classification

EDA

Thorough EDA was conducted in 'Indane Final Project EDA.ipynb' in detail.

Data Dictionary

Column	Description	Type
Lot	Name of lot of wafers processed as a batch. Numbered by process order.	int64
Slot	Physical location of wafer in a lot (1-25)	Int64
Wafer_Map	List of whether defect is present by location on wafer (2 is yes, 1 is no, 0 is background)	Object
Defect_Class	Classification of defects based on wafer map	Object

Distribution of slot values in each lot show not all lots are full lots, some are half lots with 1-12 slots (Figure 1). Slot 1 and slot 25 have lower counts than sequential slots. Edge slots are marginal/extreme processing positions in tools which cause increased defects and therefore are often empty. Average defect die impact by slot shows Slot 13 had least defects compared to other slots due to optimal processing positioning in a tool (Figure 2). Since wafers are circular, and rectangular edge dies are in radially marginal processing locations, yield is poor for edge/partial dies. An exclusion zone of 3 was created to exclude edge dies from yield impact calculations (Figure 3). Defect_Class = None identified wafers with no defect patterns and were used to determine an **8.17% baseline yield impact** which is relatively healthy but illustrates the margin for improvement. However, to truly understand manufacturing health, we need to determine how prevalent systematic defects are and quantify their impact on yield. One limitation of our analysis is that some wafers have multiple defect patterns but are classified with only one label. Therefore, our yield metrics may represent a larger impact than reality on a per defect class basis. Lot number serves as a pseudo-time axis in processing. Time paired analysis of most prevalent defect in a lot revealed Edge-Ring defects showed most prevalence in lots 26400 – 32400, so engineers should focus on tool/process changes before/after this time period to determine root cause of defects (Figure 4). Other defect patterns were prevalent across entire time range. Following page illustrates the die yield impact of each defect pattern per wafer (Figure 5). Near-full defects have highest yield impact with average of 87%. Center and Donut also affected the most dies. We need to normalize die yield impact by the number of wafers affected to understand relative priority of addressing defect issues. Priority investigations should be given to defect patterns that have

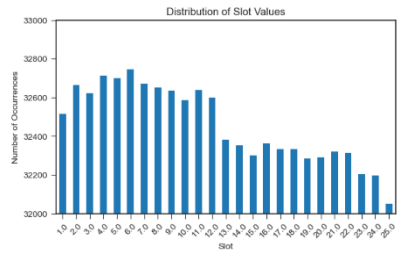


Figure 1

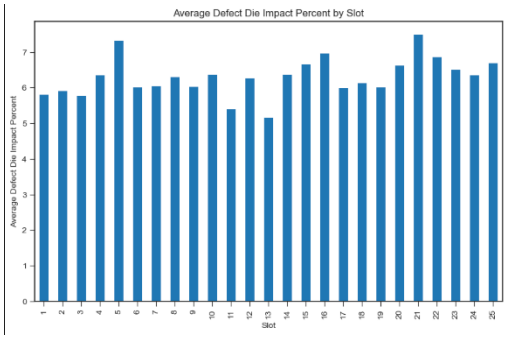


Figure 2

Figure 3

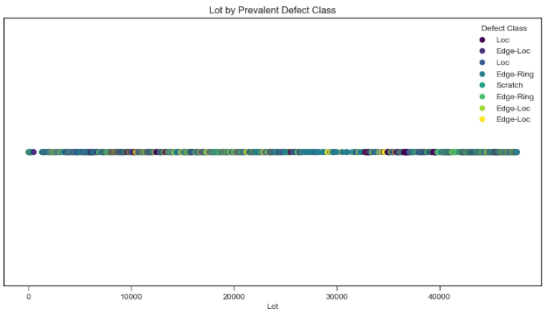


Figure 4

largest impact on yield/profit. Figure 6 shows that Center defects have the largest die impact and will require the most work/urgency to resolve. Random has a wide range suggesting more work needs to be done to classify these defects for systematic process improvements to occur. The remaining priority is as follows: Edge-Loc, Loc, Near-full, Donut. Scratch and Edge-Ring defects have lowest impact on line yield, so should be prioritized the least. Edge-Ring returns a negative value as these values were derived after subtracting the baseline 8.17% level of defects, suggesting Edge-Ring wafers have a lower defect rate across entire wafer for an unknown reason. We also notice significant class imbalance in count of wafers.

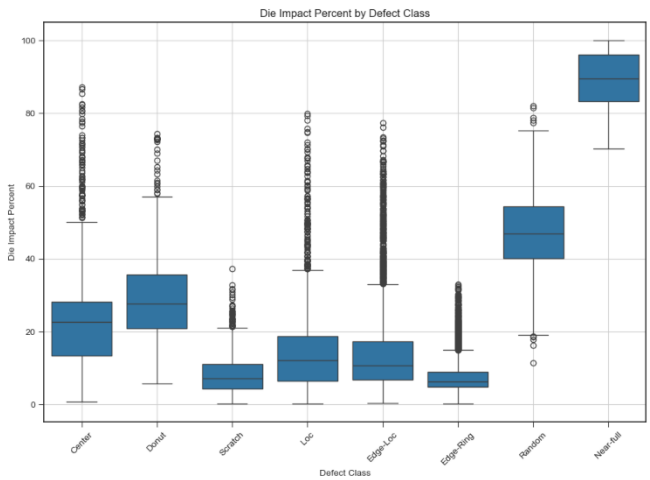


Figure 5

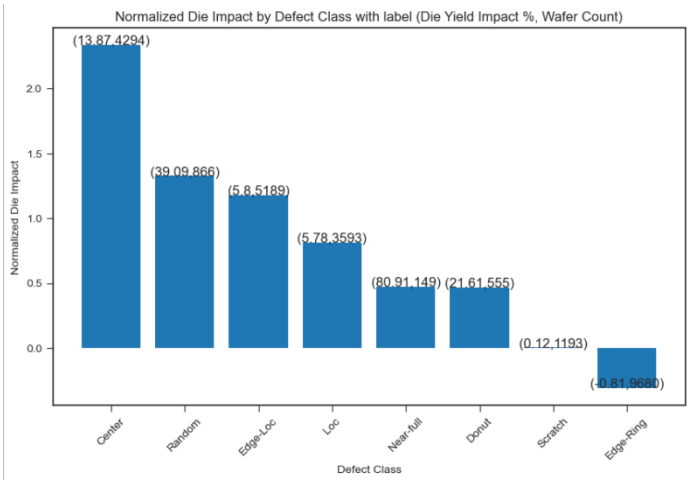
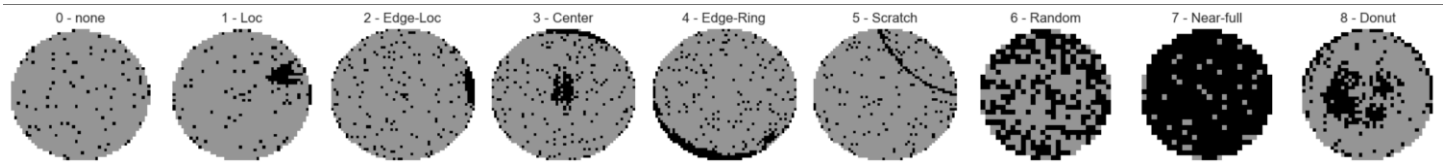


Figure 6

Data Preparation / Feature Engineering



Examples of each defect pattern are given above. To reduce noise, isolated defects were removed (set to no defect). Figure 7 shows class imbalane which was balanced with sampling reduction of some defect classes, and augmentation of others by creating new samples with random rotations of 90, 180, 270 degrees (Figure 8). A 25% validation set was excluded from class balancing. Wafer maps are in countless different dimensions, so all were transformed with nearest neighbor interpolation to be 45 x 45 which is the average wafer map dimension to reduce distortion (Figure 9). We then created several defect density features for modelling. Total defect density count was created. We also computed regional and radial defect density counts based on Figure 10. Boundaries were chosen to best capture defect classifications. We have 25 regions and 6 radial zones as features. We cannot sufficiently capture scratch defects with defect density as we need to consider directionality to characterize scratches. Deep Learning models/features will be able to classify scratches better.

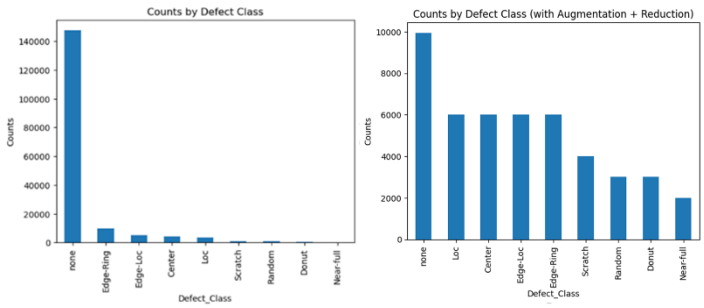


Figure 7

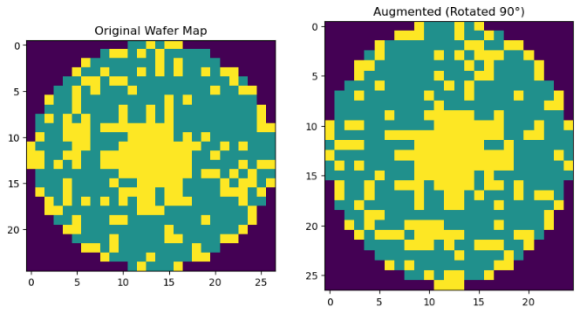


Figure 8

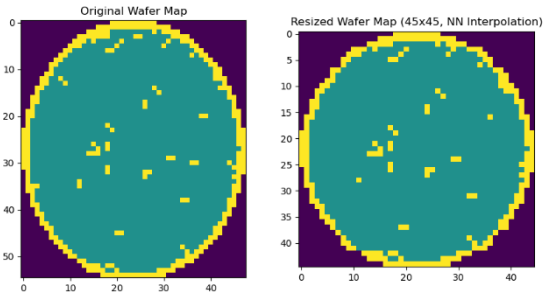
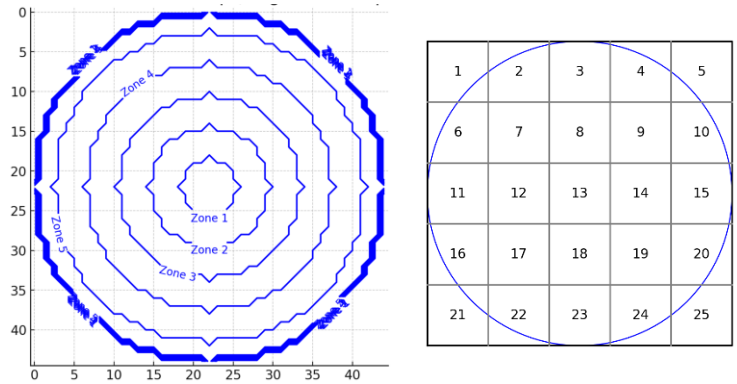


Figure 9

Figure 10



Methodology

Four models are trained on analysis: k-NN, RandomForest, CNN, and MobileNetv2. The justification for each approach is below:

1. **k-NN:** A strong intial non-parametric model to explain data with non-linear decision boundaries. Defects that are in similar locations (Center, Donut, Near-full, Edge-Ring) will be distinguishable. Edge-Loc, Loc, Scratch, will struggle as they depend on spatial relationships.
2. **RandomForest:** Robust to noisy data, allows intrepreability of feature importance and strong baseline model with quick computation time.
3. **Custom CNN:** Best performance in image classification with strong spatial relationship learning.
4. **MobileNetv2:** Lightweight model optimized for mobile devices and therefore provides strong accuracy with strong computation efficiency. It has shown to excel at wafer defect classification as one of the top performing models that preserves quick computation for real-time classification which is key in resolving manufacturing issues promptly (Tsai 2025). It will serve as our baseline model to judge our custom CNN performance.

Modelling will be done in Tensorflow 2.10 and Keras libraries in Python using GeForce NVIDIA GPU. Each model will be tuned with identical datasets of 25% stratified validation set and 75% training set. Training set will be augmented and reduced to improve model training as shown in Figure 11. This validation set will be used to measure performance with metrics, including confusion matrices, accuracy, precision, recall, F1 with the aim of powerful classification, generalizability, and speed. Precision, Recall, and accuracy will all be used to train CNN models across epochs. Due to serious class imbalance, accuracy alone will not be sufficient to validate the model.

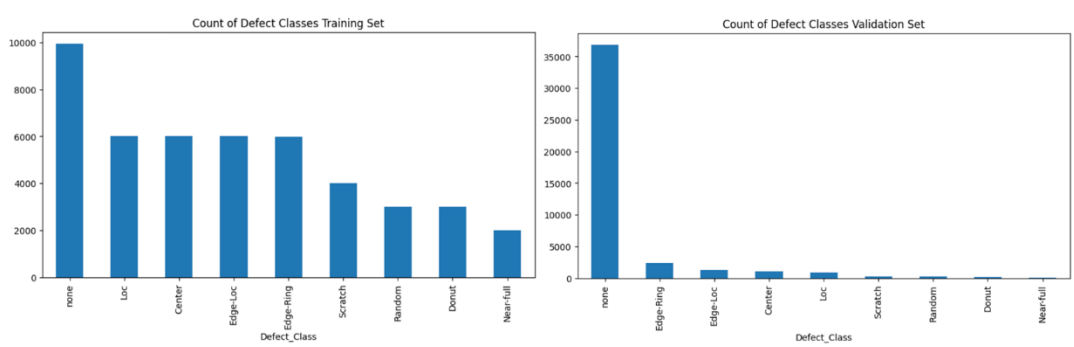


Figure 11

Model	Structure
k-NN	{'weights': 'distance', 'n_neighbors': np.int64(45), 'metric': 'manhattan'}
Random Forest	{'n_estimators': 1000, 'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'log2', 'max_depth': 70}

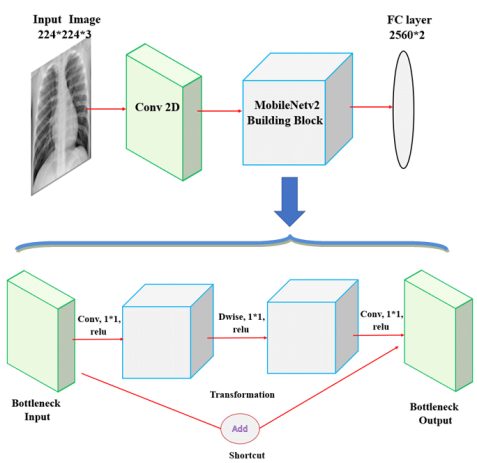
Custom CNN:

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 41, 41, 16)	416
batch_normalization (BatchNormalization)	(None, 41, 41, 16)	64
max_pooling2d (MaxPooling2D)	(None, 20, 20, 16)	0
dropout (Dropout)	(None, 20, 20, 16)	0
conv2d_1 (Conv2D)	(None, 18, 18, 64)	9280
batch_normalization_1 (BatchNormalization)	(None, 18, 18, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 9, 9, 64)	0
dropout_1 (Dropout)	(None, 9, 9, 64)	0
conv2d_2 (Conv2D)	(None, 7, 7, 128)	73856
batch_normalization_2 (BatchNormalization)	(None, 7, 7, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 3, 3, 128)	0
dropout_2 (Dropout)	(None, 3, 3, 128)	0
flatten (Flatten)	(None, 1152)	0
dense (Dense)	(None, 256)	295168
dropout_3 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 128)	32896
dropout_4 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 9)	1161

Total params: 413,609
Trainable params: 413,193
Non-trainable params: 416

We utilized a 3 block design with extensive hyperparameter tuning. Our final model included an input layer with only 16 features, which was enough to capture edges sufficiently in our 1 channel data. Kernal size of (5, 5) was used over (3, 3) due to better scratch detection with larger windows. This came at a sacrifice of lower TPR of majority class 'none'.

MobileNetv2:



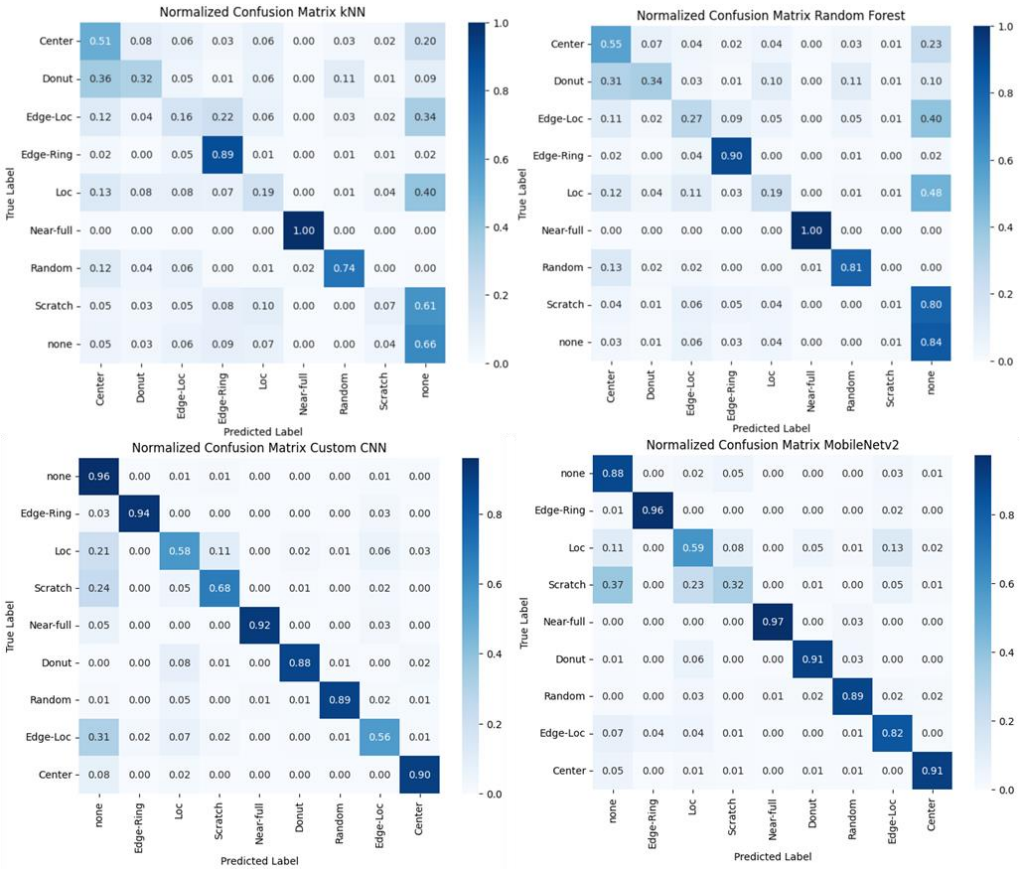
Uses 1x1 convolution windows to reduce channel numbers before applying depthwise separable convolutions, known as a bottlebeck structure. This expansion allows the model to capture more complex features, and also makes use of inverted residual training. We used this architecture but with trainable weights. GlobalAveragePooling into two dense layers with dropouts for out final sigmoid output layer of 9 classes.

Results

Model	Accuracy	Weighted Precision	Weighted Recall	Weighted F1-Score	Training Time (s)	Evalution Time (s)
k-NN	Validation: 64% Training: 97%	84%	64%	71%	6.0	2.0
Random Forest	Validation: 80% Training: 86%	87%	80%	83%	7.9	0.6
Custom CNN	Validation: 94% Training: 93%	95%	94%	94%	95.0	2.6
MobileNetv2	Validation: 88% Training: 92%	94%	88%	90%	407.2	5.9

85.3% are 'none'

Due to class imbalances, an educated guess of 'none' for all defects would obtain 85% accuracy, Therefore, k-NN is a poor overfit model. Random Forest gives better accuracy at 80% and is the quickest model with evaluation time of 0.6 seconds. However, CNNs are supreme performance in accuracy. Suprisingly, custom CNN model outperforms MobileNetv2. This pretrained model aims to classify 1,000 classes, intake three channels of data, and at least take 95x95 input shape. Our adaptation into for 9 classes with 1 channel of data with 45x45 input shape may have given worse model performance.



Confusion matrices reveal that kNN + RF models suffer identifying spatially related classes, Scratch for example. Our custom CNN provides strong performance in every defect class bucket. It is clear that MobileNetv2 struggles at identifying scratches (over Custom CNN). This could be explained by the 1x1 convolutional window in MobileNetv2, versus our 5x5 window in the first layer.

Recommendations

Recommendation is to utilize the Custom CNN model which performs the best, and has the fastest training and evalution time of both CNN models. Even with this model, we will misclassify ~25% of Loc, Scratch, and Edge-Loc defects as 'none.' Implementing this model will give strong automatic classification results and reduce human defect review efforts by 60-80%, translating to estimated annual savings between $500K and $2M (Averroes).

To further enhance classification accuracy, robustness, and scalability, we propose the following next steps and additional considerations:

1. Combining multiple models using with voting ensemble methods can further improve classification accuracy, especially for difficult-to-classify defects such as Scratch, Edge-Loc, and Loc. In this analysis, emsemble voting did not significantly improve performance at the cost of speed.
2. If we want to widely expand classes and use RGB data to uniquely identify morphologies of defects and improve classification on an individual defect level, further studies into MobileNetv2 implementation should be conducted. Deploying lightweight models on manufacturing tools or cloud-connected edge devices can be a viable option.

Integrating external datasets can help improve model robustness and address potential biases in the current dataset. The following third-party datasets can add value:

1. SEMI Semiconductor Defect Dataset: Includes real-world images of wafer defects from semiconductor fabs
2. NIST Wafer Defect Data: Provided by the National Institute of Standards and Technology, contains labeled wafer defect images and feature representations

References

Data Science Process Alliance. n.d. “CRISP-DM: A Methodology for Data Mining.” *Data Science Process Alliance*. Accessed March 15, 2025. https://www.datascience-pm.com/crisp-dm-2/.

MIR Lab. n.d. “Public Data Set.” *MIR Lab*. Accessed March 15, 2025. http://mirlab.org/dataSet/public/.

Theodosiou, T., A. Rapti, K. Papageorgiou, T. Tziolas, E. Papageorgiou, N. Dimitriou, G. Margetis, and D. Tzovaras. 2023. "A Review Study on ML-based Methods for Defect-Pattern Recognition in Wafer Maps." *Procedia Computer Science* 217: 570–583. https://doi.org/10.1016/j.procs.2022.12.253.

Gong, J., L. Chen. Wafer Map Failure Pattern Classification Using Deep Learning *CS230 Deep Learning Projects, Stanford University*. Accessed March 15, 2025. https://cs230.stanford.edu/projects_fall_2019/reports/26259703.pdf.

Li, K. S.-M., et al. 2021. "Hidden Wafer Scratch Defects Projection for Diagnosis and Quality Enhancement." *IEEE Transactions on Semiconductor Manufacturing* 34 (1): 9–16. https://ieeexplore.ieee.org/abstract/document/9273073.

Batool, Uzma, Zool Hilmi Ismail, Mohd Ibrahim Shapiai, Muhammad Tahir, Noor Jannah Zakaria, and Ahmed Elfakharany. 2021. "A Systematic Review of Deep Learning for Silicon Wafer Defect Recognition." *IEEE Access* 9: 114390–114405. https://xplorestaging.ieee.org/ielx7/6287639/9312710/09517097.pdf?arnumber=9517097&utm_source=sciencedirect_contenthosting&getft_integrator=sciencedirect_contenthosting.

Rockatoo. 2020. "WM-811K Wafer Defect Classification Using CNN-WDI." Kaggle. Accessed March 15, 2025. https://www.kaggle.com/code/rockatoo/wm-811k-wafer-defect-classification-using-cnn-wdi.

Patel, Ashish. 2019. "WM-811K Wafermap." Kaggle. Accessed March 15, 2025. https://www.kaggle.com/code/ashishpatel26/wm-811k-wafermap.

Wu, Meng-Sung. 2015. "Modeling Query-Document Dependencies with Topic Language Models for Information Retrieval." *Information Sciences* 312: 1–12. https://www.sciencedirect.com/science/article/abs/pii/S0020025515002212

Saqlain, M., B. Jargalsaikhan, and J. Y. Lee. 2019. "A Voting Ensemble Classifier for Wafer Map Defect Patterns Identification in Semiconductor Manufacturing." *IEEE Transactions on Semiconductor Manufacturing* 32 (2): 171–182. https://ieeexplore.ieee.org/document/8664594

Averroes. 2024. "Visual Inspection System Cost Breakdown & Calculating ROI." *Averroes AI Blog*, June 18, 2024. https://averroes.ai/blog/visual-inspection-system-cost-breakdown.

Tsai, T. H., and C. Y. Wang. 2025. "Wafer Map Defect Classification Using Deep Learning Framework with Data Augmentation on Imbalance Datasets." *Journal of Image and Video Processing* 2025 (6). https://doi.org/10.1186/s13640-025-00666-3.

Sandler, Mark, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. "MobileNetV2: Inverted Residuals and Linear Bottlenecks." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*: 4510–4520. https://doi.org/10.1109/CVPR.2018.00474.

Github Copilot for code completions